

Optimize Against EA

API Documentation

Developer Reference

GameLab 2 — Julius-Maximilians-Universität Würzburg

Philipp Sehne

`Philipp.Sehne@stud-mail.uni-wuerzburg.de`

David Ahrens

`David.Ahrens@stud-mail.uni-wuerzburg.de`

July 13, 2026

Contents

1	Introduction	2
1.1	Scope	2
1.2	How to read a reference entry	2
1.3	Conventions used in the codebase	2
2	Architecture Overview	3
2.1	Layering	3
2.2	Module map	3
3	Shared Infrastructure	4
3.1	Global state and settings	4
3.2	Layout components	4
3.3	Shared hooks	4
3.4	Hint system	5
3.5	Help system	5
3.6	Firebase	5
3.7	Utilities	5
4	Peak Finder Module (BattleShips)	6
4.1	Type contracts (<code>types/</code>)	6
4.2	Engine (<code>engine/</code>)	7
4.3	EA engine (<code>engine/ea/</code>)	7
4.4	Worker protocol (<code>engine/ea/ea.worker.ts</code>)	8
4.5	Hooks (<code>hooks/</code>)	8
4.6	Key components (<code>components/game/</code>)	9
5	Maze Explorer Module (mazeGame)	10
5.1	Type contracts (<code>types/</code>)	10
5.2	Engine (<code>engine/</code>)	10
5.3	Worker protocol (<code>engine/ea/maze.worker.ts</code>)	11
5.4	Hooks and components	11
6	Shooter Module (shooterGame)	12
6.1	Types and constants (<code>shooter.types.ts</code>)	12
6.2	Core loop (<code>game/core/</code>)	12
6.3	Genetic algorithm (<code>game/ga/</code>)	13
6.4	Stores	13
6.5	Horde mode (<code>horde/</code>)	13
6.6	Mods (<code>mods/</code>)	13
6.7	Hooks and components	14

7	Data Formats	15
7.1	Peak Finder map code (v1)	15
7.2	Maze code	15
7.3	Function problem code	15
7.4	Local storage keys	15
A	Extension Recipes	16
B	Tunable Constants Reference	17

Chapter 1

Introduction

1.1 Scope

Optimize Against EA is a purely client-side React 19 + TypeScript single-page application (built with Vite). It exposes no HTTP/REST endpoints; “API” in this document therefore means the **programmatic interfaces of the codebase**: the exported types, engine functions, Web Worker message protocols, React hooks, and data serialization formats a developer needs in order to extend the application or reuse its parts.

This document complements the *Handbook & User Manual*, which covers the application from the player’s perspective.

1.2 How to read a reference entry

Each documented symbol appears as an entry with its source file in the header, followed by its signature, a description, and where relevant **Params**, **Returns**, **Side effects** and **Notes** fields. All paths are relative to `OptimizeAgainstEA/OptimizeAgainstEApp/src/`.

1.3 Conventions used in the codebase

- Type-only imports use `import type {...}`.
- React components are **function** declarations; engine and utility functions are **const** arrow functions or plain functions.
- No **any**; all callback parameters are explicitly typed.
- Fitness in the map/maze EAs is *minimized* (0 = optimal); fitness in the shooter GA is *maximized*.
- Randomness is injected as `RNG = () => number` (a `Math.random`-compatible function) so engines stay deterministic under a seeded RNG.

Chapter 2

Architecture Overview

2.1 Layering

Every game module follows the same four-layer structure. Lower layers never import from higher ones:

Layer	Contents
types/ engine/	Pure type contracts and default configurations. No logic. Pure, React-free logic: problem construction, EA operators, codecs, geometry. Unit-testable in isolation.
engine/*.worker.ts	Web Workers running the EA off the main thread; communicate via typed message unions (<code>WorkerInMessage/WorkerOutMessage</code>).
hooks/ components/	React hooks bridging engine/workers to component state. Presentational and interactive React components.

2.2 Module map

Module	Route	Chapter
modules/BattleShips/	/PeakFinder	4
modules/mazeGame/	/MazeExplorer	5
modules/shooterGame/	/ShooterGame, /HordeGame	6
components/, hooks/, context/	— (shared)	3

[TODO: Architecture diagram: modules, workers, stores, and the data flow between them (one figure).]

Chapter 3

Shared Infrastructure

3.1 Global state and settings

SettingsContext / SettingsProvider	context/SettingsContext.tsx
------------------------------------	-----------------------------

Global settings provider; wraps all routes in App.tsx.

Notes. [TODO: Document the settings shape, the useSettings() hook, and persistence behaviour.]

3.2 Layout components

GameLayout, LAYOUT	components/layout/GameLayout.tsx
--------------------	----------------------------------

Three-column page shell (leftBar | canvas | sidebar) used by the shooter pages. Exports the LAYOUT constant (column widths, spacing, panel colours) for canvas-sizing arithmetic.

Notes. Pass the combined sidebar width to useScaledCanvas so the canvas scales correctly.

PageContainer	components/layout/PageContainer.tsx
---------------	-------------------------------------

[TODO: Props and usage.]

3.3 Shared hooks

useScaledCanvas (worked example)	hooks/useScaledCanvas.ts
----------------------------------	--------------------------

```
const { canvasRef, scale } = useScaledCanvas({
  baseWidth:    number,    // logical canvas width (e.g. ARENA.WIDTH)
  baseHeight:   number,    // logical canvas height
  sidebarWidth: number,    // total horizontal space taken by side panels
});
```

Scales a canvas to the largest size fitting the viewport minus the given sidebar width, preserving the logical aspect ratio.

Returns. [TODO: Confirm exact return shape.]

useViewport	hooks/useViewport.ts
-------------	----------------------

[TODO: Signature and behaviour.]

useOrientationLock	hooks/useOrientationLock.ts
--------------------	-----------------------------

[TODO: Signature and behaviour.]

3.4 Hint system

Page-wide, toggleable contextual help (components/hints/). Content lives exclusively in `hintContent.ts` (BattleShips) and `modules/mazeGame/hints/mazeHintContent.ts`.

<code>useHints</code> (worked example)	<code>components/hints/HintContext.tsx</code>
--	---

```
const { showHint, dismiss } = useHints();
showHint('vsEa.afterProbe', { vars: { gens: '5' }, actions: [...] });
```

Fires a hint by id. No-ops when hints are disabled or the hint is marked once and was already seen this session.

Side effects. Persists enabled to `localStorage` (`bs.hints.enabled`) and seen ids to `sessionStorage` (`bs.hints.seen`).

<code>HintDef</code>	<code>components/hints/hintContent.ts</code>
----------------------	--

```
{ title?, body, style: 'modal' | 'toast', once?, pauses? }
```

`pauses: true` blocks interaction behind a backdrop until dismissed.

<code>HintPopover</code>	<code>components/hints/HintPopover.tsx</code>
--------------------------	---

Anchored coach-mark wrapper. **[TODO: Props (`id`, `placement`, `show`) and the CSS-grid caveat.]**

3.5 Help system

<code>HELP_TOPICS</code> / <code>HelpButton</code>	<code>components/help/helpContent.tsx</code>
--	--

Content registry for the site-wide help modals. Adding a topic = one entry in `HELP_TOPICS` plus a `<HelpButton topic="..." />`. Each topic provides a `Gameplay` and a `Technical` tab component.

3.6 Firebase

<code>firebase</code>	<code>lib/firebase.ts</code>
-----------------------	------------------------------

[TODO: Exported firebase app/database handles; which features use them (community Raidboss); required configuration.]

3.7 Utilities

<code>rng</code>	<code>utils/rng.ts</code>
------------------	---------------------------

[TODO: Seeded RNG factory — signature and algorithm.]

Chapter 4

Peak Finder Module (BattleShips)

All paths relative to `modules/BattleShips/`.

4.1 Type contracts (`types/`)

ProblemInstance (worked example)	<code>types/map.ts</code>
----------------------------------	---------------------------

The central abstraction: game modes only ever hold a `ProblemInstance`, never a raw `MapConfig`.

```
interface ProblemInstance {
  evaluate: (x: number, y: number) => number; // 0 = global min, 1 = far away
  bounds: { xMin: number; xMax: number; yMin: number; yMax: number };
  isWin: (x: number, y: number) => boolean;
  metadata?: { name?: string;
    globalMinimum?: { x: number; y: number; value: number } };
}
```

Notes. Implemented by `createMapProblem` (`engine/functionSurface.ts`) and `createFunctionProblem` (`engine/functionProblem.ts`). New problem sources only need to produce this shape.

Coordinate, Minimum, MapConfig, ProbeResult	<code>types/map.ts</code>
---	---------------------------

[TODO: Field-by-field documentation.]

EAConfig (worked example)	<code>types/ea.ts</code>
---------------------------	--------------------------

Complete configuration of the Vs-EA opponent. `DEFAULT_EA_CONFIG` provides the defaults; `EA_PRESETS` bundles named presets for the UI.

Field	Default	Meaning
populationSize	40	Individuals per generation
maxGenerations	200	Hard stop
crossoverRate	0.8	Probability of recombination
mutationRate	0.3	Probability of mutating a child
mutationStrength	0.25	Max. perturbation (normalized coords)
mutationDecay	0.97	Per-generation strength multiplier
winPopulationFraction	0.10	Fraction inside win radius $\Rightarrow$ solved
selectionStrategy	'tournament'	'roulette' 'elitist'
crossoverStrategy	'arithmetic'	'uniform' 'singlePoint'
mutationStrategy	'gaussian'	'uniform' 'cauchy'

Individual, Generation, SelectionFn, CrossoverFn, MutationFn, RNG
types/ea.ts

[TODO: Field-by-field documentation of the EA data types and the operator function signatures.]

4.2 Engine (engine/)

createMapProblem engine/functionSurface.ts

createMapProblem(config: MapConfig): ProblemInstance — builds the smooth surface from placed minima. Tunables: DISTANCE_SCALE (0.25), LOCAL_MIN_FLOOR_MIN/MAX (0.08/0.25).
[TODO: Describe the surface formula.]

buildContours engine/contours.ts

Marching-squares isoline extraction: buildContours(evaluate, levels, resolution).
[TODO: Parameter and return documentation.]

geometry helpers engine/geometry.ts

euclideanDistance, isWithinRadius, closestMinimumDistance. [TODO: Signatures.]

sampleGradient engine/colorScale.ts

sampleGradient(t: number) — shared violet→red colour ramp used by heatmap, contours and charts.

encodeMap / decodeMap / generateRandomMap engine/mapCodec.ts

Map share-codes; see the format specification in Chapter 7.

problemCode / codeClipboard engine/problemCode.ts

[TODO: Function-problem share codes and clipboard helpers.]

4.3 EA engine (engine/ea/)

createEAS stepper (worked example) engine/ea/evolutionaryAlgorithm.ts

```
function createEAS stepper(  
  problem: ProblemInstance,  
  config: EAConfig,  
  callbacks?: EACallbacks,  
): EAS stepper
```

Creates a resumable EA. Each `stepper.step()` advances exactly one generation and returns a `StepResult` (continue / solved / exhausted). The generation loop: sort best-first → copy elite (top 5%, ≥ 1) → select parents → crossover → mutate → clamp to bounds → evaluate → win check (winPopulationFraction of the population inside the win radius).

Notes. `runEA` is the non-interactive convenience wrapper that steps to completion.

SELECTION_STRATEGIES, CROSSOVER_STRATEGIES, MUTATION_STRATEGIES
engine/ea/operators.ts

Strategy registries mapping each `*Strategy` string to its implementation. `CROSSOVER_STRATEGIES_RECORDING` variants additionally return metadata (`CrossoverResult`) consumed by the replay log. [TODO: Document each concrete operator.]

individual helpers	engine/ea/individual.ts
createRandom, evaluate, clamp. [TODO: Signatures.]	

eaReplayLog	engine/ea/eaReplayLog.ts
ReplayFrame discriminated union + buildReplayFrames. Phase order: initial → sorted → elite → breeding → mutating → newGen → winCheck. [TODO: Frame payloads per phase.]	

4.4 Worker protocol (engine/ea/ea.worker.ts)

WorkerInMessage / WorkerOutMessage (worked example)	types/ea.ts
---	-------------

The EA runs in a dedicated Web Worker. The main thread sends WorkerInMessages and receives WorkerOutMessages:

Direction	Type	Meaning
in	START	Initialize EA with MapConfig + EAConfig (does <i>not</i> start stepping)
in	STEP	Advance n generations
in	STOP	Terminate the run
out	GENERATION	Per-generation summary (best individual, stats)
out	REPLAY	ReplayFrame[] for the replay viewer

[TODO: Exact payload fields of each message variant, plus any solved/exhausted/error variants.]

4.5 Hooks (hooks/)

useEARunner (worked example)	hooks/useEARunner.ts
------------------------------	----------------------

```
const ea = useEARunner();
ea.init(mapConfig, eaConfig); // create worker, send START - does NOT run
ea.step(n);                  // advance n generations
ea.start(mapConfig, eaConfig); // run freely to completion
ea.stop(); ea.reset();
// state: ea.status, ea.generations, ea.currentGeneration,
//        ea.best, ea.totalGenerations, ea.latestReplay
```

React-side owner of the EA worker. EAStatus = 'idle' | 'running' | 'solved' | 'exhausted' | 'error'. latestReplay holds the ReplayFrame[] of the most recent step batch, consumed by useEAReplay.

useEAReplay	hooks/useEAReplay.ts
-------------	----------------------

Replay playback state machine (next, prev, play, pause). **[TODO: Full state/return documentation.]**

useGameMap	hooks/useGameMap.ts
------------	---------------------

Create-mode map state (minima, addMinimum, getCode, ...). Limits: MAX_MINIMA = 12, MIN_SPACING = 0.12. **[TODO: Full API.]**

usePlaySession	hooks/usePlaySession.ts
----------------	-------------------------

Play-mode session state (probes, status, bestProbe, probe(), reset()). **[TODO: Full API.]**

useSavedMaps	hooks/useSavedMaps.ts
--------------	-----------------------

[TODO: LocalStorage-backed saved-map CRUD.]

4.6 Key components (components/game/)

GameMap (worked example)	components/game/shared/GameMap.tsx
--------------------------	------------------------------------

The central map canvas. Important props:

```

evaluateFn?:    EvalFn           // enables ContourLayer / HeatmapLayer
revealPoints?:  Coordinate[]     // visualization only shown near these
exclusionRadius?: number         // dashed rings (create mode), normalized
defaultVizMode?: 'contour' | 'heatmap'

```

[TODO: Remaining props (click handling, markers, children).]

ContourLayer / HeatmapLayer	components/game/shared/
-----------------------------	-------------------------

Tunables: DEFAULT_CONTOUR_CONFIG (lineCount, spacingExponent, resolution),
 DEFAULT_HEATMAP_CONFIG (resolution, opacity, valueExponent). **[TODO: Props.]**

EAREplayOverlay / ReplayMap / IndividualList	components/game/vs-ea/
--	------------------------

Full-screen replay modal; population dots left, sortable individual list with role badges (ELITE / PARENT A / PARENT B / CHILD / WIN) right. Tunables: DOT_MOVE_DURATION (0.8s), autoplay dwell (1200ms). **[TODO: Props.]**

FitnessChart, ModeSelector, CodeModal, SavedMapsSidebar, SavedFunctionsSidebar	components/game/shared/
---	-------------------------

[TODO: One short entry each.]

Chapter 5

Maze Explorer Module (mazeGame)

All paths relative to `modules/mazeGame/`. The module mirrors the BattleShips structure with a *discrete* genome: an individual is a fixed-length move sequence instead of a point.

5.1 Type contracts (`types/`)

Maze core types (worked example)	<code>types/maze.ts</code>
----------------------------------	----------------------------

```
type Move = 0 | 1 | 2 | 3;           // indexes into MOVE_ARROWS / MOVE_DELTAS
type Path = Move[];                 // the genome: fixed-length move sequence
const MOVE_ARROWS = ['↑', '→', '↓', '←'];
type WallRule = 'waste' | 'break' | 'repair';
type FitnessFnId = 'manhattan' | 'geodesic' | 'length' | 'novelty';
```

Grid stores walls as bitmasks per cell (`MOVE_WALL_BIT`); `cellIndex(x, y, cols)` maps 2-D coordinates to flat indices. `WalkResult` is the phenotype: the outcome of executing a `Path` through the maze under a `WallRule`.

Notes. [TODO: Document `Cell`, `Grid`, `SerializedMaze`, `WalkResult`, `MazeProblem` field by field.]

EACConfig / DEFAULT_MAZE_EA_CONFIG	<code>types/ea.ts</code>
------------------------------------	--------------------------

Discrete-EA configuration. Strategy sets differ from BattleShips: `CrossoverStrategy = 'singlePoint' | 'uniform'`, `MutationStrategy = 'point' | 'segment'`. Genome length is auto-sized via `DEFAULT_PATH_LENGTH_FACTOR` ($3.5 \times$ the geodesic distance). [TODO: Full field table like the BattleShips one.]

5.2 Engine (`engine/`)

<code>mazeGen</code>	<code>engine/mazeGen.ts</code>
----------------------	--------------------------------

Seeded procedural maze generation — identical seed $\Rightarrow$ identical maze (the basis of the fitness-comparison experiment). [TODO: Algorithm and signature.]

<code>geodesic</code>	<code>engine/geodesic.ts</code>
-----------------------	---------------------------------

BFS distance field through corridors, precomputed at maze generation; backs the `geodesic` fitness function and genome auto-sizing. [TODO: Signature.]

<code>mazeProblem</code>	<code>engine/mazeProblem.ts</code>
--------------------------	------------------------------------

Builds a `MazeProblem` (maze + fitness function selection) analogous to BattleShips' `ProblemInstance`. [TODO: Signature and fitness-function dispatch.]

createNoveltyScorer (worked example)	engine/ea/novelty.ts
--------------------------------------	----------------------

```
function createNoveltyScorer(
  cols: number, rows: number, k: number = DEFAULT_K,
): NoveltyScorer
```

Novelty-search infrastructure: maintains an archive of behaviour descriptors (final cell reached) and scores an individual by its mean distance to the k nearest archived behaviours — higher = more novel. Used when `FitnessFnId = 'novelty'`.

Notes. **[TODO: Archive admission policy and descriptor details.]**

EA engine & operators	engine/ea/
-----------------------	------------

`individual.ts`, `operators.ts`, `evolutionaryAlgorithm.ts`, `eaReplayLog.ts` — parallel structure to Chapter 4; discrete operators (re-pick a move / splice move sequences). **[TODO: Entries per file, same pattern as BattleShips.]**

mazeCodec	engine/mazeCodec.ts
-----------	---------------------

Maze share-codes; see Chapter 7. **[TODO: Signatures.]**

5.3 Worker protocol (engine/ea/maze.worker.ts)

WorkerInMessage / WorkerOutMessage	types/ea.ts
------------------------------------	-------------

[TODO: Message table analogous to the BattleShips worker protocol.]

5.4 Hooks and components

useMazeEARunner / useMazeEAReplay / useSavedMazes	hooks/
---	--------

[TODO: Entries following the BattleShips hook pattern.]

MazeCanvas, MazeExperimentMode, MazeEASettingsPanel, MazeEAReplayOverlay, MazePathMap, MazeIndividualList	components/
---	-------------

[TODO: One short entry each (props, role).]

Chapter 6

Shooter Module (shooterGame)

All paths relative to `modules/shooterGame/`.

6.1 Types and constants (`shooter.types.ts`)

DNA (worked example)	<code>shooter.types.ts</code>
----------------------	-------------------------------

An agent genome is `number[]` of length 7; all genes in $[0, 1]$. Indices via `DNA_INDEX`:

#	Gene	Effect
0	AGGRESSION	Chase strength
1	DODGE_WEIGHT	Bullet-avoidance strength
2	SHOOT_ACCURACY	Aim precision (1 = perfect)
3	PREFERRED_RANGE	Combat distance ($\times 300 \text{ px} + 100 \text{ px}$)
4	MOVEMENT_SPEED	Speed bonus ($40 \text{ base} + \text{gene} \times 80 \text{ px/s}$)
5	PREDICT_LEAD	Aim lead factor
6	FIRE_RATE	Fire rate (higher = faster)

Related constants: `ARENA` (800×800 logical px), `GAME_CONFIG` (round duration, bullet speed, cooldowns), `STARTER_DNA`. `GamePhase` = `'idle'` | `'playing'` | `'roundEnd'` | `'evolving'`.

6.2 Core loop (`game/core/`)

<code>gameLoop.update</code>	<code>game/core/gameLoop.ts</code>
------------------------------	------------------------------------

Pure function `update(state, dt, input) → GameState`; no React dependencies. Also accumulates `PlayerGhostFrame[]` into `GameState.ghostFrames` for ghost-based evolution. **[TODO: Full signature and state transitions.]**

<code>renderer</code>	<code>game/core/renderer.ts</code>
-----------------------	------------------------------------

Canvas draw calls. **[TODO: Entry.]**

<code>vec</code>	<code>game/core/vec.ts</code>
------------------	-------------------------------

2-D vector math: `add`, `sub`, `scale`, `normalize`, `angle`, `clamp`, `perpendicular`, ... **[TODO: Signatures.]**

6.3 Genetic algorithm (`game/ga/`)

<code>evolve</code> (worked example)	<code>game/ga/evolution.ts</code>
--------------------------------------	-----------------------------------

Produces the next generation after a round: tournament selection → single-point crossover → per-gene mutation, driven by each agent's round fitness. Companions:

- `getNextAgent(population): number[]` — DNA of the next opponent to spawn;
- `presimulate(generations): Population` — round-robin pre-training (every agent vs. every other) before the first round;
- `presimulateAgainstGhost(generations, ghost)` — evolves the population against a recording of the player's previous round.

[TODO: Exact signatures and config knobs.]

<code>fitness</code>	<code>game/ga/fitness.ts</code>
----------------------	---------------------------------

`calculateFitness(stats: RoundStats): number`, `calculateRaidbossFitness(stats, roundDuration)`, `normalizeFitness(fitnesses)`. **[TODO: Scoring formulas.]**

<code>population</code>	<code>game/ga/population.ts</code>
-------------------------	------------------------------------

`initPopulation`, `updatePopulationStats`. **[TODO: Entry.]**

<code>evolution.worker.ts</code>	<code>game/ga/evolution.worker.ts</code>
----------------------------------	--

[TODO: Worker message protocol for off-thread evolution/presimulation.]

6.4 Stores

The shooter uses custom observable stores (not Zustand/Redux): components subscribe via `store.subscribe(cb)` and read `store.state` directly.

<code>gameStore</code>	<code>game/gameStore.ts</code>
------------------------	--------------------------------

Holds `GameState` outside React; consumed by `ShooterCanvas` and `DNADisplay`. **[TODO: State shape and mutation API.]**

<code>raidbossStore / analyticsStore</code>	<code>game/</code>
---	--------------------

[TODO: Entries; document the Firebase interaction of the raidboss store.]

6.5 Horde mode (`horde/`)

<code>horde subsystem</code>	<code>horde/</code>
------------------------------	---------------------

`hordeTypes.ts`, `hordeDna.ts`, `hordeMaps.ts`, `hordePathfinding.ts`, `hordeCollision.ts`, `hordeGameStore.ts`, `hordeRunStore.ts`. **[TODO: One entry per file: map format, pathfinding algorithm, wave/DNA evolution, store shapes.]**

6.6 Mods (`mods/`)

<code>mods subsystem</code>	<code>mods/</code>
-----------------------------	--------------------

`modTypes.ts`, `runModsStore.ts`, `shotEngine.ts`. **[TODO: Mod type registry and how mods alter the shot engine.]**

6.7 Hooks and components

useGameLoop / useInput / useTouchControls

hooks/

useGameLoop: requestAnimationFrame loop calling update() and driving gameStore. useInput: keyboard + mouse → InputState. useTouchControls: virtual joystick / aim zone. **[TODO: Signatures.]**

ShooterCanvas, HordeCanvas, DNADisplay, MobileJoystickZone, MobileAimZone
components/

[TODO: One short entry each.]

Chapter 7

Data Formats

Serialization formats are the closest thing the app has to a public API: they cross device boundaries via share codes and local storage.

7.1 Peak Finder map code (v1)

Produced by `encodeMap` (`BattleShips/engine/mapCodec.ts`): a URL-safe Base64 encoding of

```
{ "v": 1,                // format version
  "id": "ABC123",         // map id
  "m": [[x, y, isGlobal], ...], // minima (normalized coords)
  "wr": 0.04,            // win radius (fraction of map width)
  "t": 1699999999999 }    // timestamp
```

[TODO: Error handling of `decodeMap` on malformed/old codes.]

7.2 Maze code

`mazeGame/engine/mazeCodec.ts` + `SerializedMaze`. **[TODO: Format specification (seed-based?), versioning.]**

7.3 Function problem code

`BattleShips/engine/problemCode.ts`. **[TODO: Format specification.]**

7.4 Local storage keys

Key	Contents
<code>bs.hints.enabled</code>	Hint system on/off (<code>localStorage</code>)
<code>bs.hints.seen</code>	Seen hint ids (<code>sessionStorage</code> , per session)
[TODO: saved maps key]	Saved Peak Finder maps
[TODO: saved mazes key]	Saved mazes

Appendix A

Extension Recipes

Short how-tos for the most likely extension points. **[TODO: Flesh out each recipe with concrete steps.]**

- **Add a new analytic problem to Peak Finder** — implement a `ProblemInstance` via `createFunctionProblem` and register it.
- **Add a new EA operator** — add an entry to the relevant `*_STRATEGIES` registry and extend the strategy union type.
- **Add a new maze fitness function** — extend `FitnessFnId`, `FITNESS_FN_IDS` and `FITNESS_FN_LABELS`, implement the scorer in `mazeProblem.ts`.
- **Add a help topic** — one entry in `HELP_TOPICS` + `<HelpButton topic="..." />`.
- **Add a hint** — one entry in `hintContent.ts`, fire via `showHint(id)` or wrap with `<HintPopover>`.
- **Add a new mini-game module** — follow the four-layer module structure (Chapter 2) and register a route in `App.tsx`.

Appendix B

Tunable Constants Reference

Central table of gameplay/visual constants and their locations. **[TODO: Import the full tunables table (currently maintained in `claude.md`) and keep the two in sync — or declare one canonical.]**